

Objective

The objective of Day 9 is to develop a practical **Student Records application using Python lists**. The application stores student names and their marks in separate lists and performs different list-processing operations such as searching for a student, sorting student records according to marks, and finding the highest and lowest scores. The project helps develop a practical understanding of Python lists, indexing, loops, built-in functions, and basic data processing.

Key Features

- Stores student names and marks using Python lists.
- Allows new student records to be added.
- Displays all student records.
- Provides student name searching functionality.
- Sorts students according to their marks.
- Identifies the student with the highest score.
- Identifies the student with the lowest score.
- Validates student names and marks.
- Restricts marks to the range of 0–100.
- Provides an interactive Streamlit interface.
- Uses Streamlit session state to maintain records during the session.

Technologies Used

The project is developed using **Python and Streamlit**. Python lists are used to store student names and marks, while loops and indexing are used to access corresponding student records. Built-in functions such as `max()` and `min()` are used to identify the highest and lowest scores. The `sort()` method is used to arrange student records based on marks, while string operations are used for student searching. Streamlit provides the interactive local web interface.

Working Principle

The application maintains two synchronized lists, one containing student names and the other containing their corresponding marks. When a student record is added, the name and mark are stored at matching indexes. The application can display all records, search

for a particular student, sort records from highest to lowest marks, and calculate the highest and lowest scores. The use of matching indexes ensures that each student's name remains associated with the correct marks.

Learning Outcomes

This project provides practical experience with **Python lists, indexing, loops, searching, sorting, built-in functions, and data processing**. It also demonstrates why related data must be carefully synchronized when using multiple lists. Through the Streamlit interface, basic Python list operations are transformed into an interactive student-record application.

C:\Users\shamb\OneDrive\Desktop\VEDA internship\DAY9>

```
import streamlit as st
st.set_page_config(
    page_title="Student Records",
    page_icon="📖",
    layout="centered"
)

if "student_names" not in st.session_state:
    st.session_state.student_names = [
        "Rahul",
        "Priya",
        "Arjun",
        "Sneha",
        "Kiran"
    ]

if "student_marks" not in st.session_state:
    st.session_state.student_marks = [
        85,
        92,
        76,
        88,
        95
    ]

def add_student(name, marks):
    """Add a student name and marks to the lists."""
    if name.strip() and 0 <= marks <= 100:
        st.session_state.student_names.append(name.strip())
        st.session_state.student_marks.append(marks)
        return True
    return False

def search_student(name):
    """Search for a student by name."""
    for i, student in enumerate(st.session_state.student_names):
        if student.lower() == name.lower():
            return i
    return -1

def sort_students():
    """Sort students according to their marks."""
    records = list(
        zip(
            st.session_state.student_names,
```

```
            st.session_state.student_marks
        )
    )

    records.sort(key=lambda x: x[1], reverse=True)

    st.session_state.student_names = [
        record[0] for record in records
    ]

    st.session_state.student_marks = [
        record[1] for record in records
    ]

# -----
# Custom CSS
# -----
st.markdown("""
<style>
    .title {
        text-align: center;
        font-size: 40px;
        font-weight: bold;
    }

    .subtitle {
        text-align: center;
        font-size: 18px;
        margin-bottom: 25px;
    }

    .student-card {
        padding: 12px;
        margin: 7px 0;
        border-radius: 10px;
        border: 1px solid #dddddd;
        font-size: 17px;
    }
</style>
""", unsafe_allow_html=True)

st.markdown(
    '<div class="title">📖 Student Records</div>',
    unsafe_allow_html=True
)
```

OUTPUT: <http://localhost:8501>

 **Operations**

Select an operation:

☐  View Records

☐  Add Student

☐  Search Student

☐  Sort by Marks

☒  Highest & Lowest Score

 **Operations**

Select an operation:

☐  View Records

☐  Add Student

☐  Search Student

☒  Sort by Marks

☐  Highest & Lowest Score

 **Student Records**

Practice Python lists with student names and marks

 **Score Analysis**

 Highest Score

Kiran

95/100

 Lowest Score

Arjun

76/100

 **Student Records**

Practice Python lists with student names and marks

 **Sort Students by Marks**

Students will be arranged from highest marks to lowest marks.

Sort Students

1. Rahul — 85/100

2. Priya — 92/100

3. Arjun — 76/100

4. Sneha — 88/100

5. Kiran — 95/100

6. Abhirami — 89/100

Operations

Select an operation:

- ☐ View Records
- ☐ + Add Student
- ☒ Search Student
- ☐ Sort by Marks
- ☐ Highest & Lowest Score

Operations

Select an operation:

- ☐ View Records
- ☐ + Add Student
- ☒ Search Student
- ☐ Sort by Marks
- ☐ Highest & Lowest Score



Student Records

Practice Python lists with student names and marks



Search Student

Enter student name:

abhirami

Search

✓ Student found!

Name: Abhirami

Marks: 89/100



Student Records

Practice Python lists with student names and marks



Add Student Record

Student Name

Abhirami

Marks

89

- +

Add Student

✓ Abhirami added successfully with 89 marks.

Operations

Select an operation:

☒ View Records

☐ Add Student

☐ Search Student

☐ Sort by Marks

☐ Highest & Lowest Score

Student Records

Practice Python lists with student names and marks

Student Records

Total Students: 6

1. Rahul | Marks: 85/100

2. Priya | Marks: 92/100

3. Arjun | Marks: 76/100

4. Sneha | Marks: 88/100

5. Kiran | Marks: 95/100

6. Abhirami | Marks: 89/100

Linkedin:-<https://lnkd.in/p/d888y3bF>

Operations

Select an operation:

☐ View Records

☒ Add Student

☐ Search Student

☐ Sort by Marks

☐ Highest & Lowest Score

Student Records

Practice Python lists with student names and marks

Search Student

Enter student name:

abhirami

Search

Student found!

Name: Abhirami

Marks: 89/100

Shambhavi E L • You

Student at The Oxford College of ...

now •

Day 9 – Student Records Using Python Lists | VEDA Internship

Excited to share my Day 9 project as part of the VEDA Internship – Python Programming Track

For Day 9, I developed a Student Records... more

Like

Comment

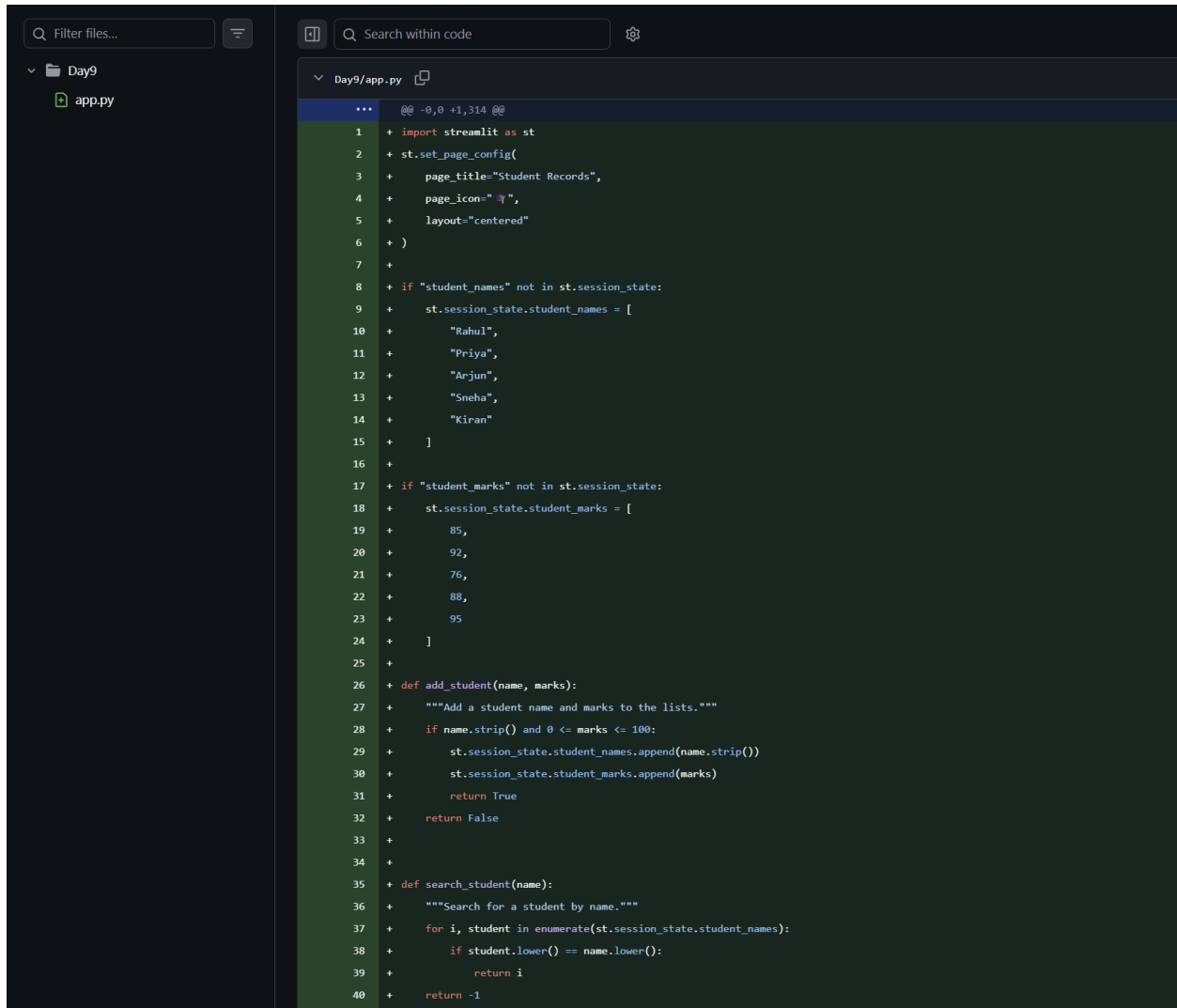
Repost

Send

Add a comment...

😊 GIF 🖼️

Github:-



```
*** @@ -0,0 +1,314 @@
1 + import streamlit as st
2 + st.set_page_config(
3 +     page_title="Student Records",
4 +     page_icon="📖",
5 +     layout="centered"
6 + )
7 +
8 + if "student_names" not in st.session_state:
9 +     st.session_state.student_names = [
10 +         "Rahul",
11 +         "Priya",
12 +         "Arjun",
13 +         "Sneha",
14 +         "Kiran"
15 +     ]
16 +
17 + if "student_marks" not in st.session_state:
18 +     st.session_state.student_marks = [
19 +         85,
20 +         92,
21 +         76,
22 +         88,
23 +         95
24 +     ]
25 +
26 + def add_student(name, marks):
27 +     """Add a student name and marks to the lists."""
28 +     if name.strip() and 0 <= marks <= 100:
29 +         st.session_state.student_names.append(name.strip())
30 +         st.session_state.student_marks.append(marks)
31 +         return True
32 +     return False
33 +
34 +
35 + def search_student(name):
36 +     """Search for a student by name."""
37 +     for i, student in enumerate(st.session_state.student_names):
38 +         if student.lower() == name.lower():
39 +             return i
40 +     return -1
```